GUÍA PARA ALUMNOS

# Calculadora en Python

Del programa a las pruebas automáticas

Programación orientada a objetos

Pruebas unitarias e integración

Primeros pasos con GitHub Actions y SonarQube

13 PASOS / 12 JOBS / EJEMPLOS REPRODUCIBLES

## Lee, ejecuta, prueba y explica.

Material de apoyo para trabajar en clase y por tu cuenta.

Requisito: Python 3.12 o posterior.

# Tu ruta de trabajo

Consulta cada paso y regresa a los ejemplos cuando lo necesites. El índice y los marcadores del PDF permiten navegar por la guía.

Paso 1 Obtener el proyecto 3

Paso 2 Ejecutar la calculadora 4

Paso 3 Encontrar la clase y el objeto 5

Paso 4 Preparar las bibliotecas de las pruebas 6

Paso 5 Ejecutar una prueba unitaria 6

Paso 6 Entender las pruebas de integración 7

Paso 7 Hacer fallar una prueba y corregirla 9

Paso 8 Medir la cobertura 9

Paso 9 Entender el workflow 10

Paso 10 Seguir cada job 11

Paso 11 Entender el orden de los jobs 13

Paso 12 Compartir un cambio y leer su resultado 14

Paso 13 Descargar el reporte y preparar la entrega 15

Preguntas para explicar lo aprendido 15

Regenerar la guía para la clase 16

# Antes de empezar

En esta práctica vas a ejecutar una calculadora, revisar su clase y comprobarla con pruebas automáticas. Después seguirás las revisiones que GitHub hace al recibir un cambio. La idea es pasar de «parece que funciona» a comprobarlo con ejemplos que cualquier compañero pueda repetir.

Empieza por los pasos 1 a 7 en tu computadora. Los siguientes explican el workflow que ya incluye el proyecto. Para usar la calculadora solo necesitas Python; las bibliotecas adicionales se instalan cuando llegues a las pruebas.

## Paso 1 Obtener el proyecto

Necesitas Python 3.12 o posterior. Comprueba la versión en una terminal:

```
TERMINAL

python --version
```

En macOS o Linux puede llamarse `python3`. En Windows también puedes consultar la versión con `py --version`.

Abre el repositorio de la calculadora. Puedes descargarlo desde **Code** → **Download ZIP** y descomprimirlo. Si ya usas Git:

```
TERMINAL

git clone https://github.com/felipecastillotecmx/calculadora.git
cd calculadora
```

Abre la carpeta en tu editor y abre una terminal dentro de ella. Debes poder ver `calculadora.py`, `requirements.txt` y la carpeta `pruebas`.

# Paso 2 Ejecutar la calculadora

Desde la carpeta del proyecto:

**TERMINAL**

```
python calculadora.py
```

Si usas `python3`, ejecuta `python3 calculadora.py`.

Prueba el promedio de tres calificaciones. Este es un ejemplo del diálogo:

**EJEMPLO DE EJECUCIÓN**

```
Elige una opción: 5
¿Cuántos números vas a usar? 3
Número 1: 8
Número 2: 9
Número 3: 10
Resultado: 9.0
¿Quieres hacer otra operación? (s/n): n
```

El programa también muestra el menú antes de pedir la opción.

| Opción | Operación | Ejemplo de números | Resultado esperado |
|---|---|---|---|
| 1 | Suma | 2, 3 y 4 | 9.0 |
| 2 | Resta | 3 y 8 | -5.0 |
| 3 | Multiplicación | 2, 3 y 0.5 | 3.0 |
| 4 | División | 9 y 2 | 4.5 |
| 5 | Promedio | 8, 9 y 10 | 9.0 |
| 6 | Potencia | 2 y 3 | 8.0 |
| 7 | Mayor y menor | 4, -2 y 8 | Mayor 8.0 y menor -2.0 |

La resta, división y potencia piden exactamente dos números y explican el orden. Las demás operaciones preguntan cuántos números usarás. Escribe los decimales con punto, por ejemplo `2.5`.

Contesta `s` para continuar o `n` para terminar. También puedes elegir `0` al ver el menú.

Ahora prueba un error: elige división e ingresa `5` y `0`. Debe aparecer un mensaje que explique que no se puede dividir entre cero, y el programa debe permitirte volver a elegir una operación. Prueba también una opción inexistente, una cantidad de cero números y una palabra donde se pide un número.

# Paso 3 Encontrar la clase y el objeto

Abre `calculadora.py`. Identifica estas partes:

| Elemento | Dónde aparece | Qué hace |
|---|---|---|
| Clase | `class Calculadora` | Agrupa los datos y las operaciones. |
| Constructor | `__init__` | Valida y guarda la operación y los números. |
| Atributos | `self.operacion` y `self.numeros` | Conservan los datos de cada objeto. |
| Métodos | `suma()`, `division()`, etc. | Realizan cada cálculo. |
| Selector de operación | `resultado()` | Usa `if` y `elif` para llamar al método adecuado. |
| Programa principal | `principal()` | Muestra el menú, usa `input()` y presenta la respuesta. |

Para probar la clase sin el menú, abre Python con `python` y escribe:

PYTHON

```
from calculadora import Calculadora

calculadora = Calculadora("promedio", [8, 9, 10])
print(calculadora.operacion)
print(calculadora.numeros)
print(calculadora.resultado())
```

Verás `promedio`, la lista `[8, 9, 10]` y `9.0`. Aquí `calculadora` es el objeto. `self` permite que sus métodos consulten los datos guardados en ese objeto.

Sal del intérprete con `exit()` antes de continuar con los comandos de terminal.

# Paso 4 Preparar las bibliotecas de las pruebas

Un entorno virtual guarda las bibliotecas de este proyecto por separado. Se crea una vez; se activa cada vez que abres una terminal nueva para trabajar.

En macOS o Linux:

TERMINAL

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

En Windows, usa **Símbolo del sistema**, también llamado **cmd**:

WINDOWS CMD

```
py -3 -m venv .venv
.venv\Scripts\activate.bat
python -m pip install -r requirements.txt
```

Desde este punto, usa `python` con el entorno activado. `requirements.txt` indica las bibliotecas y sus versiones. Entre ellas están pytest, que ejecuta las pruebas, y las herramientas de seguridad y generación del reporte.

# Paso 5 Ejecutar una prueba unitaria

Una prueba unitaria revisa una parte pequeña del programa con datos conocidos. Abre `pruebas/test_calculadora.py`. Por ejemplo:

PYTHON

```
from calculadora import Calculadora


def test_suma():
    assert Calculadora("suma", [1.5, 2, -1]).resultado() == 2.5
```

La prueba crea un objeto, obtiene el resultado y lo compara con `2.5`. `assert` exige que la comparación sea verdadera. Si no lo es, la prueba falla.

Ejecuta solo esa prueba:

TERMINAL

```
python -m pytest pruebas/test_calculadora.py::test_suma -v
```

Para ejecutar todas las pruebas de la calculadora:

```
TERMINAL

python -m pytest pruebas/test_calculadora.py -v
```

pytest encuentra las funciones cuyo nombre empieza con `test_`. `-v` muestra el nombre y el resultado de cada prueba.

También comprobamos errores esperados:

```
PYTHON

import pytest

from calculadora import Calculadora


def test_division_entre_cero():
    with pytest.raises(ValueError, match="cero"):
        Calculadora("division", [5, 0]).resultado()
```

Esta prueba pasa si la calculadora rechaza la división con un `ValueError` cuyo mensaje contiene `cero`. Que ocurra un error esperado también puede demostrar que el programa funciona bien.

# Paso 6 Entender las pruebas de integración

Una prueba de integración comprueba varias partes trabajando juntas. Abre `pruebas/test_integracion.py` y ejecútalo:

```
TERMINAL

python -m pytest pruebas/test_integracion.py -v
```

Encontrarás estos ejemplos:

- `test_programa_completo` abre la calculadora como otro proceso, le envía las respuestas del teclado y busca `Resultado: 9.0` en su salida.
- `test_menu` prueba varias conversaciones con el menú. `monkeypatch` reemplaza temporalmente `input()` por respuestas preparadas; `capsys` permite leer lo que el programa imprimió.
- `test_api` envía una operación por HTTP y comprueba la respuesta del cálculo.

La API de `api.py` permite que otro programa use la calculadora. Este ejemplo es parecido al de la prueba:

PYTHON

```
from fastapi.testclient import TestClient

from api import aplicacion

cliente = TestClient(aplicacion)
respuesta = cliente.post(
    "/calcular",
    json={"operacion": "promedio", "numeros": [8, 9, 10]},
)

assert respuesta.status_code == 200
assert respuesta.json() == {"resultado": 9}
```

`200` indica que la petición tuvo éxito. `TestClient` hace la petición dentro del proceso de prueba; no necesitas iniciar un servidor para este ejemplo. La API llama a la misma clase `Calculadora` que usa el menú.

Ejecuta ahora todas las pruebas:

TERMINAL

```
python -m pytest
```

La versión actual contiene 25 funciones de prueba. Algunas usan un bucle para revisar varios casos. Un resumen como `25 passed` significa que todas esas funciones aprobaron; el número cambiará si agregas o quitas pruebas.

`pruebas/test_reporte.py` también comprueba los PDF y la descarga de resultados. Verifica que esta guía conserve los 13 pasos, ejemplos, índice y enlaces. Sus respuestas simuladas permiten probar el reporte sin conectarse a SonarQube ni necesitar un token en tu computadora.

# Paso 7 Hacer fallar una prueba y corregirla

Trabaja localmente sobre `test_suma`. Cambia temporalmente el resultado esperado de `2.5` a `99` y ejecuta esa prueba otra vez:

```
TERMINAL

python -m pytest pruebas/test_calculadora.py::test_suma -v
```

Debe fallar: la calculadora devuelve `2.5`, no `99`. Lee el mensaje para comparar el valor obtenido y el esperado. Después restaura `2.5` y vuelve a ejecutarla.

Para agregar un caso nuevo, puedes escribir esto en `pruebas/test_calculadora.py`:

```
PYTHON

def test_suma_con_negativos():
    assert Calculadora("suma", [-3, 1]).resultado() == -2
```

Antes de compartir un cambio, restaura cualquier modificación deliberadamente incorrecta y ejecuta `python -m pytest`.

# Paso 8 Medir la cobertura

Ejecuta:

```
TERMINAL

python -m pytest --cov --cov-report=xml
```

La cobertura registra qué líneas y ramas recorrieron las pruebas. Por ejemplo, para probar la decisión de `division()` necesitamos un divisor válido y otro igual a cero. Una prueba comprueba el resultado; la otra comprueba el rechazo.

El proyecto exige una cobertura combinada de líneas y ramas de al menos 90 %. Incluye `calculadora.py`, `api.py` y `herramientas`; los archivos de prueba no se cuentan en ese porcentaje. La configuración está en `pyproject.toml`.

`coverage.xml` contiene los datos que SonarQube importa. El job de cobertura también genera `reports/junit.xml`, que describe los resultados de las pruebas. Son archivos generados, por eso no se guardan como código fuente en Git.

Tener 100 % de cobertura significa haber recorrido todo lo medido. Aun así, necesitas elegir buenos ejemplos y comprobar que sus resultados sean correctos.

# Paso 9 Entender el workflow

Abre `.github/workflows/ci.yml`. Estos conceptos te ayudarán a seguir la ejecución:

| Concepto | Qué significa en este proyecto |
|---|---|
| Workflow | La receta completa llamada **Calidad y seguridad**. |
| Job | Una revisión concreta, como `unitarias` o `secretos`. |
| Step | Un paso dentro de un job, como instalar bibliotecas. |
| Runner | La máquina temporal donde GitHub ejecuta un job. |
| Commit | Una versión de los cambios que guardas con Git. |
| Push | El envío de tus commits al repositorio remoto. |
| Pull request o PR | Una propuesta para revisar e integrar los cambios de una rama. |

`uses` llama a una acción preparada, como descargar el repositorio o configurar Python. `run` ejecuta un comando. Los tres primeros pasos se repiten porque cada job empieza en una máquina limpia.

El workflow se activa al hacer push a `main`, al abrir o actualizar un PR y cuando se ejecuta manualmente. Un push a una rama de trabajo no dispara por sí solo otra ejecución; su PR es el que activa las comprobaciones.

Una ejecución contiene varios jobs. Los independientes pueden correr en paralelo. `needs` establece qué resultados deben aprobar antes de comenzar el siguiente.

# Paso 10 Seguir cada job

## Revisión del código

El identificador es `revision`. Ejecuta `ruff check .` y `ruff format --check .`. Revisa problemas como una importación sin uso y comprueba el formato. Por ejemplo, una línea mal indentada puede necesitar corrección antes de continuar. No cambia los archivos automáticamente.

## Pruebas unitarias

El identificador es `unitarias`. Ejecuta las pruebas de la calculadora y del reporte. Por ejemplo, exige que el promedio de `[8, 9, 10]` sea `9`. Si una comparación falla, este job queda en rojo.

## Pruebas de integración

El identificador es `integracion`. Espera a `unitarias` y ejecuta las pruebas del menú y la API. Por ejemplo, verifica que escribir una opción y sus números termine mostrando el resultado correcto.

## Cobertura del proyecto

El identificador es `cobertura`. Espera a `unitarias` e `integracion`, vuelve a ejecutar todas las pruebas y produce el XML de cobertura. Se hace una ejecución completa para reunir las mediciones en un solo archivo. Si la cobertura total queda por debajo del 90 %, falla aunque las comparaciones hayan pasado.

## Contraseñas y secretos

El identificador es `secretos`. Gitleaks revisa el historial de Git buscando credenciales. Por ejemplo, puede detectar una clave de API que alguien haya pegado por error. Los valores detectados se censuran en el reporte. No escribas credenciales reales para probar este job.

## Seguridad estática

El identificador es `seguridad`. Bandit lee el código Python y busca patrones riesgosos. Por ejemplo, puede advertir sobre el uso de `eval()` con datos recibidos del usuario. No necesita poner a funcionar la API para analizarla.

## Vulnerabilidades de dependencias

El identificador es `dependencias`. pip-audit consulta avisos de seguridad para las bibliotecas del proyecto y sus dependencias. Por ejemplo, una versión con una vulnerabilidad conocida puede hacer fallar el job. El reporte ayuda a identificar qué paquete necesita atención.

## Dependencias obsoletas

El identificador es `obsoletas`. Comprueba si hay versiones más recientes de las bibliotecas y guarda una lista. Por ejemplo, puede mostrar una versión instalada y otra disponible. Esa diferencia es informativa y no bloquea por sí misma el workflow; un error al ejecutar la herramienta sí puede bloquearlo.

## OWASP ZAP

El identificador es `zap`. Espera a integración, secretos, seguridad estática y auditoría de dependencias. Inicia la API y la prueba mientras funciona, usando su descripción `openapi.json`. Por ejemplo, revisa respuestas y cabeceras de seguridad. En este proyecto los avisos que detecta ZAP hacen fallar el job.

La API existe temporalmente dentro del runner. Este job no despliega la calculadora en Internet ni analiza servidores ajenos al proyecto.

## Análisis SonarQube con cobertura

El identificador es `sonarqube`. Espera a cobertura, revisión del código, secretos, seguridad estática y auditoría de dependencias. Envía el código y `coverage.xml` al servidor. Por ejemplo, muestra la cobertura y los issues abiertos del proyecto.

Un **issue** es un hallazgo sobre el código; no es lo mismo que una prueba fallida. Este job envía el análisis y el job siguiente consulta su evaluación. En el servidor Community de este proyecto solo se analiza `main`.

## Reporte y Quality Gate

El identificador es `reporte`. Espera a `sonarqube`, descarga sus resultados y genera PDF, JSON y Markdown. El **Quality Gate** es el conjunto de condiciones que SonarQube exige para aprobar el análisis. Por ejemplo, una condición puede limitar los issues nuevos o exigir una cobertura mínima del código nuevo.

El PDF distingue todo el proyecto del código nuevo definido por la línea base del servidor. Código nuevo no significa necesariamente solo el último commit. `N/D` significa que no hay una medición disponible, no que el resultado sea cero.

Si el Quality Gate falla, el job queda en rojo y conserva el reporte para poder revisar la causa. Su descarga también requiere permiso de lectura en SonarQube.

Cuando aprueba el Quality Gate, el mismo job regenera esta guía y la publica como **guia-para-alumnos-pdf**, lista para descargar y compartir en clase.

## Resumen final

El identificador es `resumen`. Espera a todos los demás y utiliza `if: always()` para revisar sus resultados incluso si alguno falla. Si `unitarias` falla, por ejemplo, el resumen también marca que la ejecución no fue aprobada. Para corregirla debes empezar por el fallo original.

En los PR, `sonarqube` y `reporte` aparecen omitidos porque el análisis se reserva para `main`. El resumen permite esas omisiones esperadas.

# Paso 11 Entender el orden de los jobs

Estas son las dependencias exactas de la versión actual:

| Job | Debe esperar a |
|---|---|
| `revision` | Ninguno. |
| `unitarias` | Ninguno. |
| `secretos` | Ninguno. |
| `seguridad` | Ninguno. |
| `dependencias` | Ninguno. |
| `obsoletas` | Ninguno. |
| `integracion` | `unitarias`. |
| `cobertura` | `unitarias`, `integracion`. |
| `zap` | `integracion`, `secretos`, `seguridad`, `dependencias`. |
| `sonarqube` | `cobertura`, `revision`, `secretos`, `seguridad`, `dependencias`. |
| `reporte` | `sonarqube`. |
| `resumen` | Todos los demás. |

Por eso no todos comienzan al mismo tiempo. Si falla un requisito, sus jobs dependientes normalmente se omiten. El resumen sí se ejecuta para explicarlo.

# Paso 12 Compartir un cambio y leer su resultado

Si tienes permisos en el repositorio donde trabajas, crea una rama para tu práctica. Para compartirla por primera vez, el recorrido básico es:

**TERMINAL**

```
git checkout -b practica-pruebas
python -m pytest
git add pruebas/test_calculadora.py
git commit -m "Agrega una prueba de suma con negativos"
git push -u origin practica-pruebas
```

El ejemplo supone que agregaste la prueba del paso 7. Si descargaste un ZIP, trabaja primero en una copia configurada con Git por tu docente o clona el repositorio donde tengas permisos.

En GitHub abre un **pull request** para proponer el cambio. En **Actions**, abre la ejecución de **Calidad y seguridad** y revisa los jobs. Para inspeccionar un fallo, abre el primer job en rojo y después el paso que falló.

| Resultado | Cómo interpretarlo |
|---|---|
| Success o verde | El control terminó y aprobó. |
| Failure o rojo | Abre el registro para encontrar la causa. |
| Queued | Espera una máquina o capacidad de ejecución. |
| Skipped | No se ejecutó por una condición o una dependencia. |

Si falla una prueba, compara el resultado obtenido con el esperado. Si falla el formato, revisa con `python -m ruff check .` y aplica formato con `python -m ruff format .`. Si fallan las dependencias o la seguridad, atiende el hallazgo que indica el reporte y vuelve a probar.

Después de corregir, crea otro commit y haz push a la misma rama. El PR volverá a ejecutar los controles. Cuando el cambio llegue a `main`, su push ejecutará también SonarQube y la generación del PDF.

# Paso 13 Descargar el reporte y preparar la entrega

Abre una ejecución de `main` que ya haya terminado. Ve a **Actions** → **ejecución** → **Artifacts** y descarga **sonarqube-report-pdf-json-md**. Descomprime el archivo:

- `sonarqube.pdf` presenta el resumen, las métricas y la trazabilidad del análisis.
- `sonarqube.json` conserva los datos para consultarlos desde otro programa.
- `sonarqube.md` ofrece un resumen que puedes leer como texto.

Comprueba que el PDF corresponde al commit de tu entrega. Revisa el Quality Gate, la cobertura general y nueva, y los issues de ambos alcances.

Para compartir esta guía, descarga **guia-para-alumnos-pdf** desde la misma ejecución, o abre el PDF del repositorio. La guía y el reporte de SonarQube son dos documentos con propósitos distintos.

Para una copia nueva del repositorio, quien configure el workflow debe guardar `SONAR_TOKEN` en **Settings** → **Secrets and variables** → **Actions**. Debe ser un **User Token** con permisos **Browse** y **Execute Analysis** sobre el proyecto `calculadora` de <https://sonarqube.lynvera.ai>. El token no se incluye en los archivos ni se necesita para ejecutar las pruebas locales.

Entrega `calculadora.py` y el PDF solicitado. No agregues `.venv`, credenciales, archivos de cobertura ni reportes del análisis al commit; `.gitignore` ya excluye los archivos temporales. La guía de clase en `material/` sí se guarda en el repositorio para que todos puedan descargarla.

# Preguntas para explicar lo aprendido

1. ¿Qué datos guarda cada objeto `Calculadora`?
2. ¿Qué método decide qué operación ejecutar?
3. ¿Por qué la prueba de división entre cero debe pasar cuando aparece un error?
4. ¿Qué partes se conectan en una prueba de integración?
5. ¿Qué diferencia hay entre cobertura alta y resultados correctos?
6. ¿Qué diferencia hay entre una biblioteca obsoleta y una vulnerable?
7. ¿Por qué ZAP necesita una API funcionando y Bandit no?
8. ¿Por qué aparecen varios jobs dentro de una sola ejecución?
9. ¿Qué sucede con `integracion` si falla `unitarias`?
10. ¿Dónde descargas el PDF y cómo compruebas qué commit analizó?

# Regenerar la guía para la clase

El texto de esta guía está en `GUIA_ALUMNOS.md`. Después de editarlo, activa el entorno virtual con las bibliotecas instaladas y ejecuta:

```
TERMINAL

python -m herramientas.guia_pdf
```

Se actualiza `material/GUIA_ALUMNOS.pdf`, incluido su índice navegable. Abre el archivo y revisa las páginas antes de compartirlo. Si cambias el código de los ejemplos, ejecuta también las pruebas para mantenerlos al día.

El proyecto se distribuye con la licencia MIT; puedes consultar sus condiciones en `LICENSE`.